

Can LLMs Improve Accessibility? Evaluating LLM Detection and Remediation of WCAG Violations in Web Dashboards

Maitri Pathak
Virginia Tech
Blacksburg, VA, USA
maitripathak14@vt.edu

Yoongi Baek
Virginia Tech
Blacksburg, VA, USA
ybaek01@vt.edu

Hanru Li
Virginia Tech
Blacksburg, VA, USA
hanru@vt.edu

Jason Kayne
Virginia Tech
Blacksburg, VA, USA
jkayne@vt.edu

Sehrish Basir Nizamani
Virginia Tech
Blacksburg, VA, USA
sehrish@vt.edu

Abstract

Web accessibility remains difficult to maintain in modern web dashboards because many accessibility barriers are tied not only to visible layout, but also to code-level semantics such as labels, link purpose, and alternative text. Manual expert review is reliable but time-consuming, whereas rule-based tools may miss context-dependent problems or offer limited remediation guidance. This study evaluates whether large language models (LLMs) can support accessibility work by detecting and repairing intentionally injected WCAG 2.1 AA violations in synthetic web dashboard interfaces. We tested three LLMs—ChatGPT, Claude, and Gemini—across six violation categories: low color contrast, missing non-text alternatives, missing form input labels, empty buttons, missing ARIA (Accessible Rich Internet Applications) labels, and empty links. The dataset contained 60 dashboard variants, and each variant was evaluated with two prompt styles—a broad context-based review prompt and a structured step-by-step reasoning prompt—producing 360 model-prompt outputs. Outputs were scored using a 15-point rubric that covered false positives, detection accuracy, implementation accuracy, code reasoning, and whether the violation persisted after the proposed fix. Results show that Claude achieved the highest mean total score ($M = 12.00$, $SD = 1.93$), followed by ChatGPT ($M = 11.58$, $SD = 2.01$) and Gemini ($M = 10.97$, $SD = 2.41$); a one-way ANOVA found a statistically significant model effect ($F = 7.152$, $p = .0009$). Structured prompting also improved performance, increasing mean score from 11.21 to 11.83 ($p = .0001$). These findings suggest that LLMs can be useful early-stage accessibility assistants, especially for visible or syntactically clear violations. Still, they should not replace expert validation for semantic and context-dependent accessibility decisions.

CCS Concepts

• **Human-centered computing** → **Accessibility technologies**; **Empirical studies in accessibility**.

Keywords

Accessibility, Large Language Models, WCAG, Web Dashboards, Automated Evaluation, Code Repair

1 Introduction

Accessible web design is a central concern in Human-Computer Interaction (HCI) because inaccessible interfaces can prevent users

with disabilities from completing ordinary digital tasks. The Web Content Accessibility Guidelines (WCAG) 2.1 define testable success criteria for making web content more accessible, including requirements for contrast, text alternatives, keyboard access, labels, and programmatic names [14]. However, accessibility problems remain common in practice. Large-scale accessibility reports continue to identify recurring errors such as low-contrast text, missing alternative text, empty links, empty buttons, and missing form labels across many public websites [11]. These same issues are especially prevalent in dashboard interfaces, where dense visual information, icons, charts, tables, and controls are arranged in compact layouts that complicate both manual review and automated scanning.

Traditional accessibility evaluation usually combines automated testing tools and human expert review. Automated tools such as Axe and WAVE are useful because they quickly identify many rule-based violations. Still, they cannot fully assess user intent, visual context, or whether a proposed repair actually preserves an interface’s meaning [4, 11]. Human review is more context-aware, but it requires time, expertise, and repeated inspection as an interface changes. This creates a practical gap for students, designers, and developers who need faster feedback during prototyping.

Recent LLMs create a new opportunity for accessibility support by enabling them to read source code, interpret screenshots, explain interface problems, and generate code changes. At the same time, LLM-generated code can be incomplete, overconfident, or semantically incorrect, especially when prompts do not constrain the model’s reasoning process [3, 10]. Therefore, this study asks whether LLMs can serve as useful accessibility assistants for web dashboards, and under what prompting conditions they perform better.

We investigate two research questions:

- **RQ1:** How effectively can LLMs detect, repair, and justify common WCAG-related accessibility violations in synthetic web dashboards, and what patterns emerge across violation types and models?
- **RQ2:** Does a structured, step-by-step prompt improve LLM accessibility detection and remediation compared with a broad general prompt?

Our contribution is a controlled, dashboard-specific evaluation of three LLMs across six WCAG violation types, comparing broad

and structured prompting strategies. Unlike prior work that focuses on either detection or code generation in isolation, our five-dimensional rubric jointly assesses detection accuracy, implementation correctness, code reasoning quality, false positive rate, and whether violations are fully resolved after the proposed fix. This design allows us to distinguish models that identify issues reliably from those that also produce trustworthy, minimal repairs.

2 Related Work

2.1 Web Accessibility and Automated Auditing

WCAG provides an internationally used framework for accessible web content, including success criteria related to non-text content, color contrast, labels, link purpose, and name/role/value semantics [14]. Automated accessibility tools are widely used because they can quickly scan rendered pages or code for common accessibility failures. For example, Axe-core describes itself as a rule-based accessibility testing engine that covers WCAG 2.0, 2.1, and 2.2 criteria [4]. These tools are valuable for catching many recurring violations, but automated detection cannot prove a page is fully accessible, and some problems require human judgment about content, context, and task flow [11]. Recent research suggests LLMs may help address these limitations by evaluating WCAG criteria that typically require manual inspection. One study found that an LLM-based approach substantially outperformed existing automated tools for success criteria involving non-text content and link purpose, areas where rule-based systems often struggle [7]. This suggests that LLMs may complement conventional scanners by supporting accessibility checks that depend more heavily on interpretation and context.

2.2 LLMs for Code Generation and Repair

LLMs are increasingly used to generate and modify code, but prior work has shown that generated code can include subtle correctness, security, or quality problems [3]. Accessibility adds another layer of difficulty because correct code must not only compile or look visually similar but also provide meaningful semantics to assistive technologies. Recent accessibility-focused work suggests that LLMs can generate accessible code for basic cases but may struggle with more complex semantic requirements, such as ARIA usage [10]. Recent work has also evaluated whether LLM-generated interfaces meet accessibility standards in practice. One study found that LLM-generated user interfaces often reproduced accessibility barriers even when accessibility was explicitly requested in prompts, with stronger performance on visible issues such as color contrast and weaker performance on semantic accessibility requirements such as labeling and element relationships [6]. The findings suggest that LLM-generated code may still require human accessibility review despite improvements in prompting. Complementary work has explored iterative repair pipelines combining automated scanners with LLM-based fixes, substantially reducing violation counts versus prompting-only baselines such as contextual, zero-shot, and ReAct prompting [5], while related work on LLMs as automated heuristic evaluators found GPT-4o moderately consistent for issue detection but more variable on severity judgments, even without domain-specific fine-tuning [9].

This motivates evaluating LLMs not only as code generators, but as accessibility reviewers and repair tools, and assessing whether structured prompting can improve repair quality.

2.3 Prompt Structure in LLM Evaluation

Prompt design can substantially shape LLM output quality by directing the model’s attention and requiring explicit intermediate reasoning steps. Chain-of-thought prompting, which asks models to reason step-by-step before producing an answer, has been shown to improve performance on complex tasks requiring multi-step reasoning [12]. For accessibility work, this matters because a broad prompt may encourage useful observations but also invite hallucinated issues. In contrast, a structured prompt can require the model to inventory interface elements, map issues to specific WCAG criteria, and self-check its proposed fix before finalizing a response. These techniques are consistent with general prompt-engineering strategies such as those cataloged by White et al. [13]. Prompt sensitivity is a known challenge in LLM evaluation: small changes in wording can alter which issues a model flags and how confidently it reports them [8]. Our experiment directly compares broad and structured prompting approaches in a controlled accessibility setting, allowing us to isolate the effect of prompt design on detection, repair, and reasoning quality.

3 Methodology

3.1 Dataset

We created a controlled dataset of 60 synthetic web dashboard variants using Figma Make, an AI-powered interface prototyping tool, to generate the initial interface code, which was then manually reviewed and modified to ensure each dashboard contained exactly one intentionally injected accessibility violation. Beyond this single injected violation, individual dashboards varied in overall layout and component composition (e.g., which combination of cards, charts, tables, or navigation elements were present), so that each of the 60 variants represented a distinct dashboard design rather than a copy of a single template with only the violation changed.

The dashboards represented common analytics interfaces containing cards, charts, tables, navigation controls, icons, links, and form inputs. Violations were injected by directly modifying the source code of each variant—for example, removing `<label>` elements and replacing them with placeholder text only, replacing button text content with empty strings, or setting foreground colors to produce a contrast ratio below the WCAG 1.4.3 threshold (below 4.5:1 for normal text and below 3:1 for large text). Each dashboard variant contained a single violation category, allowing model outputs to be evaluated against a known target issue.

The six violation categories were selected because they represent the most frequently observed accessibility failures in large-scale web audits [11], and each maps to a testable WCAG criterion:

- (1) **Color Contrast:** low-contrast text or icons that reduce readability for users with low vision (WCAG 1.4.3).
- (2) **Non-text Content:** missing, generic, or incorrect alternative text for images, icons, or chart content (WCAG 1.1.1).
- (3) **Missing Form Input Labels:** inputs relying on placeholders or visual context without programmatic labels (WCAG 1.3.1 and 3.3.2).

- (4) **Empty Buttons:** button elements with no visible text or accessible name (WCAG 4.1.2).
- (5) **ARIA (Accessible Rich Internet Application) Labels:** icon-only interactive controls lacking meaningful accessible names (WCAG 4.1.2).
- (6) **Empty Links:** links with unclear or missing link purpose (WCAG 2.4.4).

For each violation category, we evaluated ten dashboard variants. Each variant was tested with three LLMs and two prompt styles, resulting in $60 \times 3 \times 2 = 360$ scored outputs.

3.2 Models and Prompt Conditions

We compared three LLMs: ChatGPT 5.2, Claude Sonnet 4.6, and Gemini 3.0 Thinking. Each model received a screenshot of the dashboard and the full source code, and was asked to identify accessibility issues and propose minimal code-level repairs. We used two prompt conditions.

The second prompt was designed to reduce unsupported claims by making the model move from interface inventory to rule verification before proposing code edits. Because Figma Make was used to generate dashboard code, some dashboards may have contained accessibility issues beyond the single injected violation, which could affect false positive scoring.

3.3 Scoring Rubric

Ground truth for each dashboard was established at the point of violation injection: because the research team deliberately introduced exactly one known accessibility violation into each dashboard’s source code (Section 3.1), the type and location of that violation served as the ground truth against which LLM outputs were compared, rather than being independently derived through a separate rater review process. Each output was evaluated by a single trained rater using a 15-point rubric with five criteria scored from 0 to 3, who checked the model’s response against this known ground truth. Higher scores indicate better performance.

- **False Positives (0–3):** whether the model avoided hallucinated or unsupported issues. A score of 3 indicates no false positives; 0 indicates more than three non-issues flagged or harmful code changes suggested.
- **Detection Accuracy (0–3):** whether the model identified the intentionally injected violation. A score of 3 indicates all injected issues were detected; 0 indicates the violation was missed or misidentified.
- **Implementation Accuracy (0–3):** whether the proposed code fix was technically correct and WCAG-aligned. A score of 3 indicates a fully compliant fix; 0 indicates no fix or an incorrect fix.
- **Code Reasoning (0–3):** whether the explanation connected the code change to the relevant accessibility rule and the specific line of code changed. A score of 3 indicates a logical, line-specific justification; 0 indicates a nonsensical or hallucinated explanation.
- **Violation Presence (0–3):** whether the original violation appeared to be resolved after the proposed fix. A score of 3 indicates no issues remain; a score of 0 indicates multiple issues persist.

It is important to clarify the comparison structure of this study. Human evaluators did not independently detect or repair the injected violations as a parallel condition; the LLMs performed the detection-and-repair task, and a single human rater then scored each LLM output against the known injected violation using the rubric above. Accordingly, a “good” score in this study indicates that a model successfully detected and remediated the injected violation without introducing new accessibility problems, not that the model outperformed a human accessibility reviewer completing the same task. We did not include a human-performed detection/repair condition; we return to this in Section 6.

This rubric was chosen because a model can identify the right issue but still produce a weak or incomplete fix. Separating detection, implementation, reasoning, and the presence of remaining violations allowed us to evaluate the practical usefulness of the generated repair. Because a single rater scored each dashboard, inter-rater reliability could not be measured; this is discussed further in the Limitations section.

The Python analysis pipeline (statistical functions and chart formatting) was developed with generative AI assistance; all outputs were verified by the authors.

4 Results

4.1 Overall Model Performance

Table 1 summarizes the average scores by model. Claude had the highest overall mean score ($M = 12.00$, $SD = 1.93$), followed by ChatGPT ($M = 11.58$, $SD = 2.01$) and Gemini ($M = 10.97$, $SD = 2.41$). A one-way ANOVA showed a statistically significant model effect on total score ($F = 7.152$, $p = .0009$). Claude also had the highest rate of perfect scores, with 17 of 120 outputs scoring 15/15 (14.2%), compared with 11 of 120 for ChatGPT (9.2%) and 7 of 120 for Gemini (5.8%). Notably, all 35 perfect scores across all three models occurred exclusively in the Color Contrast and Non-text Content categories, with no perfect scores recorded for any other violation type—a pattern that reflects the relative difficulty of semantic violations discussed in Section 4.3.

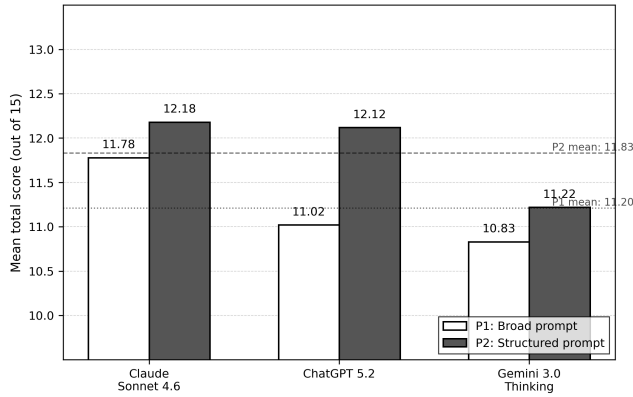
The models were generally strong at detecting the injected target issue: all three had mean detection accuracy above 2.7/3. The larger differences appeared in implementation accuracy, code reasoning, violation resolution, and false-positive avoidance. Claude achieved the highest Code Reasoning score (2.67 vs. 2.28 for ChatGPT and 2.22 for Gemini). However, Gemini achieved the highest FP Avoidance score (1.27), followed by ChatGPT (1.17) and Claude (1.02). Thus, overall repair performance and conservative issue reporting did not follow the same model ranking. Gemini’s substantially higher standard deviation ($SD = 2.41$ vs. 1.93 for Claude) reflects inconsistent performance: Gemini performed competitively on simpler violations but occasionally failed to produce any corrected code on harder ones, resulting in scores as low as 5/15 on individual outputs. This suggests that the main challenge was not always recognizing that an accessibility problem existed, but producing a precise, minimal, and trustworthy repair.

4.2 Prompt Comparison

Structured prompting improved the average total score across all models. Prompt 1 produced a mean score of 11.21 ($SD = 2.38$), while

Table 1: Mean evaluation scores by model. Metrics scored 0–3 (higher = better); total out of 15, computed from unrounded values.

Model	FP Avoidance	Detection Acc.	Implementation Acc.	Code Reasoning	Violation Presence	Total Score
Claude	1.02	2.88	2.70	2.67	2.76	12.00 (SD 1.93)
ChatGPT	1.17	2.84	2.62	2.28	2.62	11.58 (SD 2.01)
Gemini	1.27	2.77	2.34	2.22	2.37	10.97 (SD 2.41)

**Figure 1: Mean total scores by model and prompt condition. Structured prompting (P2) improved performance across all three models compared with the broad prompt (P1), with the largest gain observed for ChatGPT 5.2.****Table 2: Prompt-level comparison across all models.**

Prompt	FP Avoidance	Det.	Impl.	Reason	Viol.	Total
Broad (P1)	0.96	2.74	2.56	2.39	2.57	11.21
Structured (P2)	1.34	2.93	2.56	2.39	2.61	11.83

Prompt 2 produced a mean score of 11.83 ($SD = 1.89$). A paired t-test comparing matched model–dashboard–violation outputs showed that this difference was statistically significant ($t = -4.055$, $p = .0001$). The improvement was approximately 5.5% relative to the Prompt 1 mean, and the lower standard deviation for Prompt 2 indicates more consistent outputs and higher average scores.

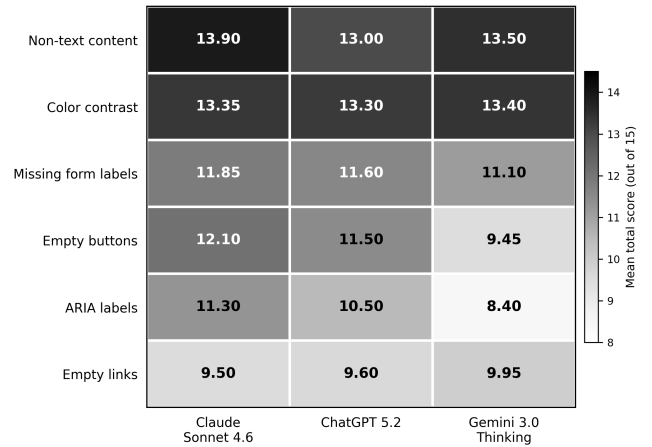
Examining the sub-criteria in Table 2 reveals an important nuance: structured prompting improved both detection accuracy (2.74 to 2.93) and FP Avoidance (0.96 to 1.34). Because higher FP Avoidance scores indicate fewer unsupported findings, this suggests that the structured checklist helped models identify the target issue more consistently and reduced false-positive behavior. In contrast, implementation accuracy (2.56 to 2.56) and code reasoning (2.39 to 2.39) were identical across both prompt conditions, indicating that structured prompting did not yield better code repairs or clearer justifications.

4.3 Performance by Violation Type

Table 3 shows the mean total score for each violation category. The highest-scoring categories were Non-text Content ($M = 13.47$) and

Table 3: Mean Total Scores by Violation Type and Model. Scores out of 15.

Violation Type	Claude	ChatGPT	Gemini	Overall
Non-text Content	13.90	13.0	13.50	13.47
Color Contrast	13.35	13.30	13.40	13.35
Missing Input Label	11.85	11.60	11.10	11.52
Empty Buttons	12.10	11.50	9.45	11.02
ARIA Labels	11.30	10.50	8.40	10.07
Empty Links	9.50	9.60	9.95	9.68

**Figure 2: Mean total scores by violation type and model. All models performed best on Non-text Content and Color Contrast violations, while Empty Links and ARIA Labels produced the lowest scores overall. Gemini showed the largest variability across violation categories.**

Color Contrast ($M = 13.35$). These issues were comparatively easy because the expected fixes were often syntactically straightforward: adding or correcting alternative text, hiding decorative icons from assistive technologies, or changing explicit color values to improve contrast ratios.

The trends shown in Figure 2 reinforce that semantic accessibility issues, such as Empty Links and ARIA Labels, were substantially more difficult for all models than visually explicit violations.

The lowest-scoring category was Empty Links ($M = 9.68$), followed by ARIA Labels ($M = 10.07$). These issues required more semantic judgment. For an empty link, the model often had to infer the link’s purpose from the surrounding layout and code structure.

For ARIA label problems, the model had to decide whether visible text already provided an accessible name or whether an icon-only control required an explicit label. Importantly, all three models scored similarly on Empty Links (9.50–9.95), suggesting this difficulty was not model-specific but reflected an inherent challenge in inferring link purpose without runtime context.

Gemini showed the largest variation across violation types: it scored competitively on Non-text Content ($M = 13.50$, the highest among models in that category). Still, it dropped sharply on ARIA Labels ($M = 8.40$) and Empty Buttons ($M = 9.45$), categories in which it occasionally failed to generate corrected code at all. This within-model inconsistency, reflected in Gemini’s higher overall standard deviation, suggests that Gemini’s performance depends more heavily on violation type than the other two models.

5 Discussion

5.1 What LLMs Did Well

The models were useful at quickly identifying common accessibility issues and suggesting plausible code changes. Their strongest performance appeared in categories where the violation had a clear, syntactic mapping from problem to repair. For example, low-contrast text can often be improved by changing explicit color values, and missing non-text alternatives can be repaired by adding descriptive alt text or marking decorative images as hidden from assistive technology. In these cases, the LLMs functioned like flexible accessibility assistants: they flagged the problem, explained the user impact, and generated a targeted fix. Although Claude achieved the highest overall mean score, Gemini received the highest FP Avoidance score (1.27), followed by ChatGPT (1.17) and Claude (1.02). This result shows that the model producing the strongest overall repairs was not necessarily the model reporting the fewest unsupported issues. In practical workflows, developers may therefore need to balance repair quality against the review burden created by additional findings.

5.2 Where LLMs Were Less Reliable

The lower scores for Empty Links and ARIA Labels show that LLMs still struggle when the correct answer depends on context rather than syntax. A model may add an ARIA label that sounds plausible but does not reflect the actual action, or it may fail to recognize that a link needs a meaningful purpose beyond simply having a href attribute. Some outputs also included unnecessary changes, such as editing layout wrappers or adding attributes to elements that were not the source of the violation—behaviors that conflict with the goal of minimal, testable repairs.

Gemini’s pattern deserves particular attention: it scored highest among all models on Non-text Content. Still, it dropped sharply on Empty Buttons and ARIA Labels, at times returning no corrected code at all. This inconsistency, reflected in its higher standard deviation, suggests that Gemini is more sensitive to the type of violation than the other models. Practitioners using Gemini for accessibility review should be aware that its reliability may vary substantially depending on the class of problem being evaluated.

Related work on accessibility remediation in mobile applications reports similar challenges. Prior research found that LLMs could often identify accessibility concerns and generate syntactically valid

fixes, while still failing to resolve the underlying violation or provide sufficient WCAG-grounded justification [1]. Although that work focused on mobile interfaces rather than dashboards, it reinforces the distinction between identifying accessibility problems and producing reliable repairs that our rubric was designed to capture.

5.3 Prompting Implications

The structured prompt improved overall performance, particularly detection accuracy. However, implementation accuracy and code reasoning scores were identical across both prompt conditions—the structured checklist helped models identify the correct issue more often. Still, it did not yield better code repairs or more precise justifications. This is a meaningful finding for practitioners: structured prompting reduces the chance that a violation will be missed, but developers should not assume that a step-by-step prompt will also yield a higher-quality fix. Human review of the generated code remains necessary regardless of prompt style.

For developers integrating LLMs into accessibility workflows, our findings suggest a useful division of labor: use structured prompts during early prototyping to quickly surface potential violations, and reserve expert review for semantic and context-dependent issues such as ARIA labeling and link purpose. The prompt does not replace expertise, but it can make LLM output more auditable by forcing the model to name specific elements and WCAG criteria before proposing changes.

6 Limitations

This study used synthetic dashboards rather than deployed production systems. Synthetic data enabled controlled comparisons across models and violation types. Still, it may not capture the complexity of real applications, including routing, state changes, authentication, asynchronous loading, and dynamic screen-reader announcements. Additionally, this study did not include a condition in which human evaluators independently detected and repaired the same injected violations; the human rater’s role was limited to scoring LLM outputs against the known ground-truth violation. As a result, our results speak to whether LLMs can reliably detect and remediate WCAG violations on their own, not to how LLM performance compares with unassisted human performance on the same task; establishing such a comparison is an important direction for future work.

The scoring process focused on code-level outputs rather than live testing with assistive technologies. In addition, each dashboard evaluation was reviewed by a single rater, which may introduce individual interpretation bias when assessing LLM-generated repairs. Future studies could strengthen reliability by having multiple reviewers independently evaluate each output and report inter-rater agreement. The research team developed the scoring rubric itself and did not validate it against an external ground truth or an existing accessibility evaluation instrument. Together, the reliance on synthetic dashboards and a single rater means that the reported scores should be interpreted as evidence of what is achievable under controlled conditions rather than as a generalizable estimate of LLM accessibility performance on real, deployed interfaces evaluated by multiple independent reviewers.

Finally, because LLM products evolve quickly and all models were tested at a single point in time with default settings, the exact model rankings should be treated as a snapshot rather than a permanent conclusion. Prompt sensitivity—how much performance changes with minor rewording of the same prompt type—was not evaluated and represents an additional source of variance not captured in this study.

7 Future Work and Impact

Near-term extensions (next 12 months). We plan to extend this work in four concrete directions. First, we will replicate the study on *deployed* open-source dashboards drawn from public GitHub repositories, replacing Figma Make-generated interfaces with production code that includes routing, state management, and asynchronous behavior. Second, we will pair LLM repairs with automated accessibility scanners (Axe and WAVE) and live screen-reader testing (NVDA, VoiceOver) to verify whether the model’s claimed fixes hold up under assistive-technology evaluation, not just static code review. Third, we will introduce multi-rater scoring with three independent reviewers per output and report inter-rater agreement using Cohen’s κ , addressing the single-rater limitation of the present study. Fourth, building on recent work showing that persona-conditioned LLM agents broaden issue coverage in usability testing [2], we will test whether assigning agents specific accessibility personas—screen-reader users, low-vision users, keyboard-only users, users with cognitive accessibility needs—changes which violations the model surfaces and repairs, and whether persona diversity reduces the contextual blind spots we observed for Empty Links and ARIA Labels.

Long-term impact (5–25 years). Accessibility remains a chronic, under-prioritized concern: WebAIM’s annual audits show that most of the top one million home pages fail basic WCAG checks year after year, despite decades of guideline publication [11]—the bottleneck is practice, not knowledge. Over the next five to ten years, we expect accessibility evaluation to move from post-hoc specialist audits into the everyday development loop, the way linters do today; our finding that structured prompts improve detection but not repair quality suggests tooling should separate the *flagging* step (where LLMs can be trusted) from the *fixing* step (where they still need oversight). Over a longer horizon, LLM-based agents with diverse accessibility personas could simulate interaction failures during prototyping rather than after deployment, though this will require validating simulated behavior against real disabled users and keeping them in control of design decisions as agentic tooling matures.

Contribution to the broader HCI community. Beyond these findings, we contribute a reusable evaluation infrastructure—the 60-dashboard corpus, five-dimensional rubric, and broad-vs-structured prompt pair—that other researchers can adopt to benchmark new models and prompting strategies as the field evolves.

8 Conclusion

This study shows that LLMs can support early-stage accessibility evaluation of web dashboards, particularly for common, syntactically clear WCAG violations. Across 360 scored outputs, Claude achieved the highest overall mean score, while Gemini achieved the

highest FP Avoidance score. All three models demonstrated useful detection ability.

Structured prompting significantly improved detection consistency, though it did not improve implementation accuracy or the quality of code reasoning. Semantic violations such as Empty Links and ARIA Labels remained difficult for all models, and Gemini showed notably higher performance variance across violation types. Related work on LLM consistency in automated usability evaluation has also found that even when the same model is queried multiple times under identical conditions, severity ratings can diverge meaningfully across sessions [9], suggesting that single-session scoring—whether by a human rater or by the model itself—should be interpreted with caution. Therefore, LLMs should be used as assistive tools to accelerate accessibility reviews and guide early-stage code repairs, not as replacements for expert evaluation or user-centered accessibility testing.

References

- [1] Wajdi Aljedaani, Ahmed Aljohani, Marcelo M. Eler, Abdulrahman Habib, and Hyunsook Do. 2025. LLMs for Accessibility in Mobile Apps: Detection and Repair. In *Proceedings of the 22nd International Web for All Conference (W4A '25)*. Association for Computing Machinery, New York, NY, USA, 172–183. doi:10.1145/3744257.3744270
- [2] Yoongi Baek, Calvin Tozer, Aarya Ghimire, and Sehrish Basir Nizamani. 2026. Persona Diversity: Comparing Usability-Testing Outcomes Across Heterogeneous Personas. In *Artificial Intelligence in HCI*, Helmut Degen and Stavroula Ntoa (Eds.). Springer Nature Switzerland, Cham, 262–274.
- [3] Chun Jie Chong, Zhihao Yao, and Iulian Neamtii. 2024. Artificial-Intelligence Generated Code Considered Harmful: A Road Map for Secure and High-Quality Code Generation. arXiv:2409.19182 [cs.CR] <https://arxiv.org/abs/2409.19182>
- [4] Deque Systems. [n. d.]. *Axe-core: Open Source Accessibility Engine for Automated Testing*. <https://www.deque.com/axe/axe-core/>
- [5] Nadeen Fathallah, Daniel Hernández, and Steffen Staab. 2025. AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '25)*. Association for Computing Machinery, New York, NY, USA, Article 88, 22 pages. doi:10.1145/3663547.3746360
- [6] Alexandra-Elena Gurita and Radu-Daniel Vatavu. 2025. When LLM-Generated Code Perpetuates User Interface Accessibility Barriers, How Can We Break the Cycle?. In *Proceedings of the 22nd International Web for All Conference (W4A '25)*. Association for Computing Machinery, New York, NY, USA, 124–134. doi:10.1145/3744257.3744266
- [7] Juan-Miguel López-Gil and Juanan Pereira. 2024. Turning manual web accessibility success criteria into automatic: an LLM-based approach. *Univers. Access Inf. Soc.* 24, 1 (March 2024), 837–852. doi:10.1007/s10209-024-01108-z
- [8] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. 2024. State of What Art? A Call for Multi-Prompt LLM Evaluation. *Transactions of the Association for Computational Linguistics* 12 (2024), 933–949. doi:10.1162/tacl_a_00681
- [9] Nolan Platt, Ethan Luchs, and Sehrish Nizamani. 2025. Catching UX Flaws in Code: Leveraging LLMs to Identify Usability Flaws at the Development Stage. In *2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 152–158. doi:10.1109/VL-HCC65237.2025.00024
- [10] Hyunjae Suh, Mahan Tafreshipour, Sam Malek, and Iftekhar Ahmed. 2025. Human or LLM? A Comparative Study on Accessible Code Generation Capability. arXiv:2503.15885 [cs.SE] <https://arxiv.org/abs/2503.15885>
- [11] WebAIM. 2026. *The WebAIM Million: The 2026 Report on the Accessibility of the Top 1,000,000 Home Pages*. <https://webaim.org/projects/million/2026>
- [12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
- [13] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs (Monticello, IL, USA) (PLoP '23)*. The Hillside Group, USA, Article 5, 31 pages.
- [14] World Wide Web Consortium (W3C). 2018. *Web Content Accessibility Guidelines (WCAG) 2.1*.